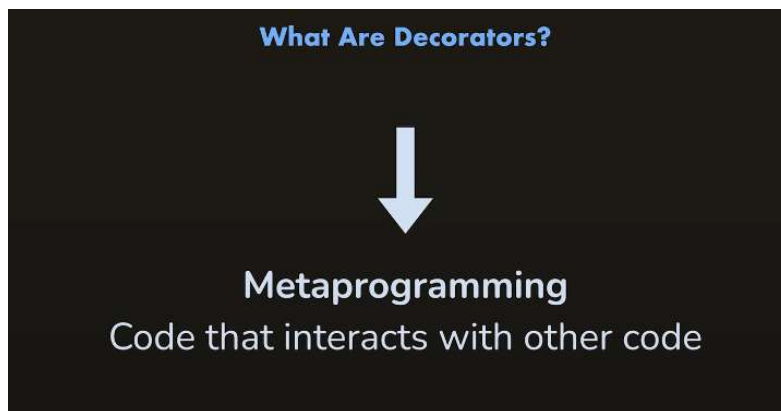




What Are Decorators? And ECMAScript Decorators vs Experimental Decorators



So what are decorators and what's the idea behind them? Well, decorators, not just in TypeScript, but in programming in general, because this feature exists in other programming languages as well, are a metaprogramming feature.

And metaprogramming is simply code that you write that interacts with other code, and that other code then makes up the actual application,

And typically decorators look something like this. For example, this is how they are used in TypeScript. They start with an @ symbol or you have to add such an @ symbol in front of the decorator name. They may take arguments, but they don't have to. And they can then be attached to things, so for example, to a property of a class here.

```

    IsInt,
    Length,
    IsEmail,
    IsFQDN,
    IsDate,
    Min,
    Max,
  } from 'class-validator';

export class Post {
  @length(10, 20)
  title: string;

  @contains('hello')
  text: string;

  @IsInt()
  @Min(0)
  @Max(10)
  rating: number;

  @IsEmail()
  email: string;

  @IsFQDN()
  site: string;

  @IsDate()
  createDate: Date;
}

let post = new Post();
post.title = 'Hello'; // should not pass
post.text = 'this is a great post about hell world'; // should not pass
post.rating = 11; // should not pass
post.email = 'google.com'; // should not pass
post.site = 'googlecom';

```

what kind of things you can attach decorators to later.

that the decorator that's attached to a thing changes that thing. For example, here, this length decorator is coming from a third party library called Class Validator, and it adds validation logic to the property

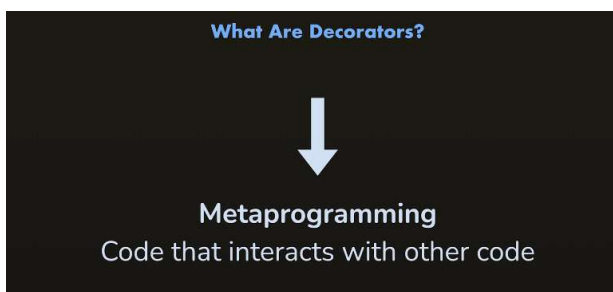
to which it is added, ensuring that this title in this class has a certain min and max length.

But you can, of course, implement any kind of logic with decorators.

The core idea, however, and that's important, is that decorators are a feature that allow you to write code that changes the behavior of other code. Like here, where a decorator changes the behavior of the title property.

The end. The idea behind decorators is that you write code in a certain way so that you can alter the behavior of some other code to which you attach these decorators.

That's the idea.



There are two different approaches or ways of writing decorators, of building them in TypeScript.

Because in TypeScript, you can build both ECMAScript and experimental decorators.

And both kinds of decorators can be built with TypeScript. The difference is that the ECMAScript decorators, the official decorators, if you want to call 'em like this,

are actually also a JavaScript feature.

So you could build them without TypeScript as well.

You don't need TypeScript to build ECMAScript decorators.

TypeScript Supports Two Kinds Of Decorators

JS

ECMAScript Decorators

Built-into JavaScript

Can be used without TypeScript

TS

Experimental Decorators

Previously planned implementation

Did not make it into JavaScript

Only supported by TypeScript

Requires "experimentalDecorators"
configuration

Currently when I'm recording this, it is a stage three proposal, which in the end means that the proposal has been recommended for implementation, so that browser engines and all the other engines that in the end execute JavaScript code should support that feature, you could say

or should support it in the future And once that's the case you'll be able to build such decorators with just JavaScript without TypeScript. But TypeScript also supports this feature, and it does so already actually. So even before all the different engines implement this feature,

and for the time being, also depending on your compilation target set in the TS config file, TypeScript will simply compile this decorator code to code that will work in older engines, other browsers and so on as well.

So you can use this feature today with TypeScript at some point in the future. You can also theoretically use it without TypeScript. And then there are these experimental decorators, and these decorators are built with a different syntax and they're only available in TypeScript.

These decorators simply reflect an older version of that implementation proposal for decorators. So a version that did not make it to stage three, you could say, a version that's not going to be implemented in plain Vanilla JavaScript.

But TypeScript started supporting this old proposal many, many years ago already, and it still does and will for the foreseeable future. But this proposal did not make it into JavaScript, that's why it's only available in TypeScript.

And that's why in TypeScript, you can build both kinds of decorators with their difference syntaxes because TypeScript does support both. However, if you do want to build such experimental decorators, you must tweak your TS config, and you must make sure that the experimental decorators flag is set to true there.

That's important. But we'll cover experimental decorators, so this older approach, as you could say, in a separate section in this course, and it is definitely still a feature that's worth learning because all existing projects that use decorators in TypeScript will be using that old approach.

But again, we do cover that in a separate section. In this section here, I'll get you started with the EXMAScript decorators, and I'll introduce you to them, show you how you can build decorators and how you can use them

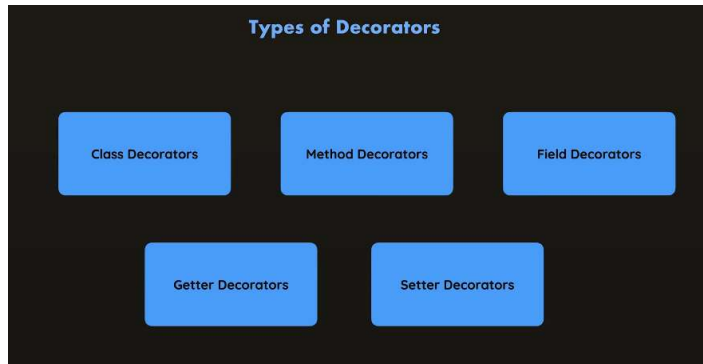
Exploring Different Types of Decorators

Now, before we get started writing decorator code,

I also briefly want to introduce you to the different types of decorators. So not the different kinds,

the two different syntaxes you can use, but different types of decorators. And here it's important to understand that this decorator's feature is an object-oriented programming feature,

which means this decorator's feature, the decorators you are going to build, can only be used in conjunction with classes, their methods, their fields, or getters and setters in the end.



Building a First Decorator

Decorator is an object oriented related feature, I will start by adding a class

```
1
2
3 class Person {
4   name = 'Max';
5
6   greet() {
7     console.log('Hi, I am ' + this.name);
8   }
9 }
```

And now let's build a decorator, let's say a decorator

that should be attached to the entire class

to interact with it, to interact with that code,

if you wanna call it like this.

And for that, we create a function

because decorators in JavaScript

are just functions.

but it is a function written in a certain way

receiving a certain amount of arguments and so on.

But it is just a function in the end.

And I'll name my function logger here,

but the name is up to you.

```
function logger() {}  
  
class Person {  
  name = 'Max';  
  
  greet() {  
    console.log('Hi, I am ' + this.name);  
  }  
}
```

because I plan on creating a decorator that can be attached

to this class here

to log information about it, just to get started.

So we can try attaching this function

as a decorator to this class.

And how do we do that?

Well, by adding something in front of this class,

```
function logger() {}  
  
@  
class Person {  
  name = 'Max';  
  
  greet() {  
    console.log('Hi, I am ' + this.name);  
  }  
}
```

by adding the @ symbol.

That's an important symbol.

```

1 function logger() {}
2
3 @
4 class Person {
5   name = 'Max';
6
7   greet() {
8     console.log('Hi, I am ' + this.name);
9   }
10 }

```

You have to use the @ symbol in order to add a decorator

to a class, a field, a method, or a getter or a setter.

After @ symbol, you put the name of the function

you want to use as a decorator,

```

function logger() {}

@logger
class Person {
  name = 'Max';

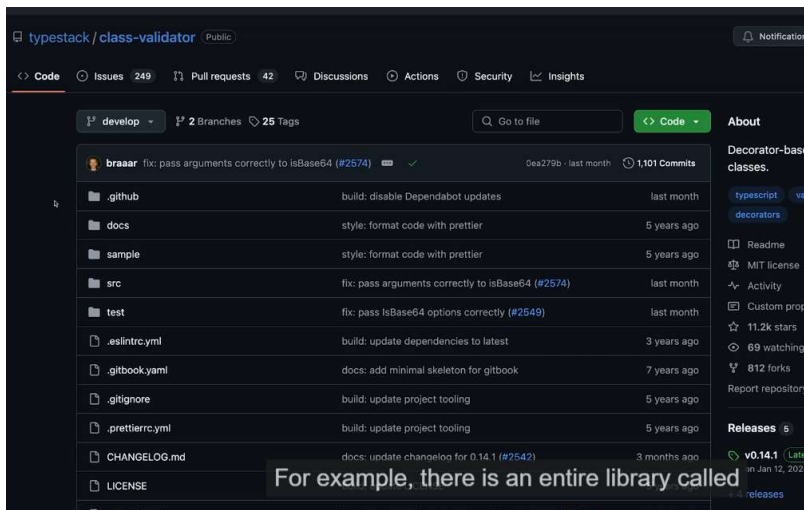
  greet() {
    console.log('Hi, I am ' + this.name);
  }
}

```

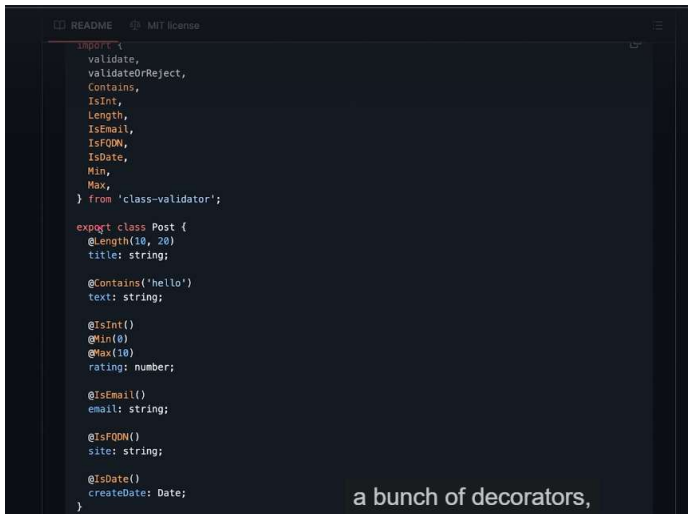
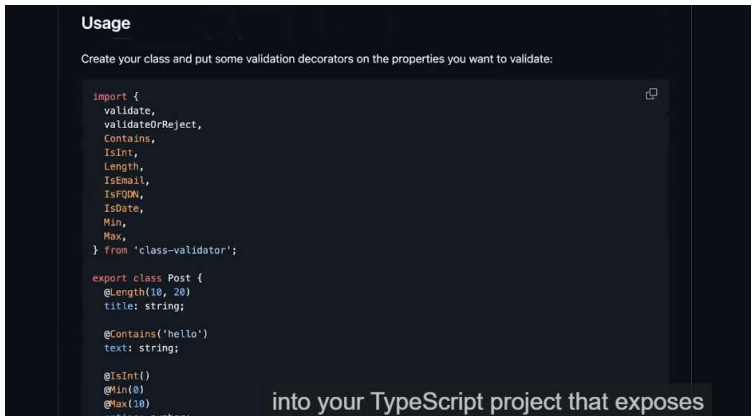
like logger here.

So it's code interacting with our code.

It's the logger function here, in this case,



class validator, which you could install



```

    Min,
    Max,
  } from 'class-validator';

  export class Post {
    @Length(10, 20)
    title: string;

    @Contain('hello')
    text: string;

    @IsInt()
    @Min(0)
    @Max(10)
    rating: number;

    @IsEmail()
    email: string;

    @IsFQDN()
    site: string;

    @IsDate()
    createDate: Date;
  }

  let post = new Post();

```

which you can add to class properties, for example,

to add validation to those properties

You can use this feature to attach logic to your code.

```

1  function logger() {}
2
3  @logger

```

'logger' accepts too few arguments to be used as a decorator here. Did you mean to call it first and write '@logger()'? ts(1329)

function logger(): void

[View Problem \(↵F8\)](#) [Quick Fix... \(⌘.\)](#)

```

9
10 }

```

but it is not a valid decorator function.

```

1  function logger() {}
2
3  @logger
4  class Person {
5    name = 'Max';
6
7    greet() {
8      console.log('Hi, I am ' + this.name);
9    }
10 }

```

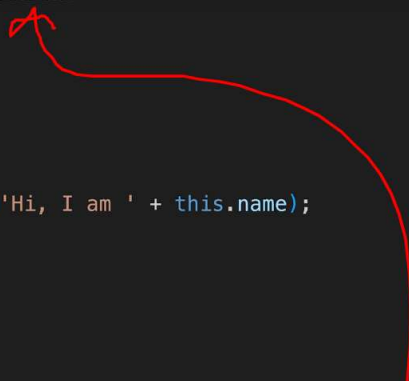
and we need to accept more arguments here.

For an ECMAScript decorator, it's two arguments.


```
function logger(target) {}

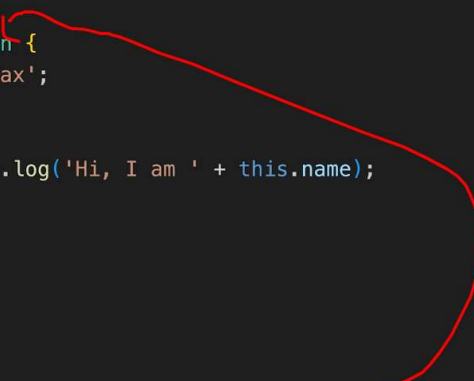
@logger
class Person {
  name = 'Max';

  greet() {
    console.log('Hi, I am ' + this.name);
  }
}
```



And the first argument is the target of the decorator,

```
1 function logger(target) {}
2
3 @logger
4 class Person {
5   name = 'Max';
6
7   greet() {
8     console.log('Hi, I am ' + this.name);
9   }
10 }
```



the thing you're attaching it to.

```
@logger
class Person {
  name = 'Max';

  greet() {
    console.log('Hi, I am ' + this.name);
  }
}
```

So in this case, this class.

```
function logger(target: {}) {}

@logger
class Person {
  name = 'Max';

  greet() {
    console.log('Hi, I am ' + this.name);
  }
}
```

Now, the type of target is a bit more complex

```
function logger(target: any) {}

@logger
class Person {
  name = 'Max';

  greet() {
    console.log('Hi, I am ' + this.name);
  }
}
```

for the moment, I'll simply set it to any

```
function logger(target: any) {}

@logger
class Person {
  name = 'Max';

  greet() {
    console.log('Hi, I am ' + this.name);
  }
}
```

Now as you can see, the error went away,

but actually you typically accept two arguments

because the second argument is a context object

that gives you extra information about the thing

you're attaching the decorator to.

The context is an object with information about that thing.

And that context object is of type class decorator context.

```
decorators.ts U ●
1 function logger(target: any, ctx: ClassDecoratorContext) {}
2
3 @logger
4 class Person {
5     name = 'Max';
6
7     greet() {
8         console.log('Hi, I am ' + this.name);
9     }
10 }
```



A type that's built into TypeScript.

```
decorators.ts U ●
1 function logger(target: any, ctx: ClassDecoratorContext) {
2
3 }
4
5 @logger
6 class Person {
7     name = 'Max';
8
9     greet() {
10         console.log('Hi, I am ' + this.name);
11     }
12 }
```



So that is now how we can define a valid class decorator.

```

decorators.ts U
1 function logger(target: any, ctx: ClassDecoratorContext) {
2   console.log(target);
3   console.log(ctx);
4 }
5
6 @logger
7 class Person {
8   name = 'Max';
9
10  greet() {
11    console.log('Hi, I am ' + this.name);
12  }
13 }

```

to get an idea of what's inside of these two arguments.

```

ts decorators.ts U X
1 function logger(target: any, ctx: ClassDecoratorContext) {
2   console.log('logger decorator');
3   console.log(target);
4   console.log(ctx);
5 }
6
7 @logger

```

PROBLEMS DEBUG CONSOLE PORTS COMMENTS OUTPUT TERMINAL

```

$ node decorators.js
logger decorator
[class Person]
{
  kind: 'class',
  name: 'Person',
  metadata: undefined,
  addInitializer: [Function (anonymous)]
}
$

```

Building a Class Decorators That Edits a Class

```

TS decorators.ts X
1 function logger(target: any, ctx: ClassDecoratorContext) {
2   console.log('logger decorator');
3   console.log(target);
4   console.log(ctx);
5 }
6
7 @logger
8 class Person {
9   name = 'Max';
10
11  greet() {
12    console.log('Hi, I am ' + this.name);
13  }
14 }

```

but what else can we do with that logger decorator here?

Well, you can also use decorators

to change the thing you are attaching them to.

So to change the class, for example,

```
1 function logger(target: any, ctx: ClassDecoratorContext) {
3   console.log(target);
4   console.log(ctx);
5
6   return
7 }
8
9 @logger
10 class Person {
11   name = 'Max';
12
13   greet() {
14     console.log('Hi, I am ' + this.name);
15   }
16 }
```

a new class that's based on the old class in the end,

and you can do that with a syntax that might look strange,

but that is valid JavaScript code.

```
log('Hi, I am ' + this.name);
You can return an anonymous class
```

and then "extends target."

```
le.log('Hi, I am ' + this.name);
and I'm not giving my class a name here
```

g(' because it's an anonymous class,

which I'm just creating temporarily to return it,

Decorator function return type 'typeof (Anonymous class)' is not assignable to type 'void | typeof Person'.
 Type 'typeof (Anonymous class)' is not assignable to type 'typeof Person'.
 Type '(Anonymous class)' is missing the following properties from type 'Person': name, greet ts(1270)

```
function logger(target: any, ctx: ClassDecoratorContext): typeof (Anonymous class)
```

[View Problem](#) (⌘F8) No quick fixes available

```
11 @logger
12 class Person {
13   name = 'Max';
14
15   greet() {
```

a pretty complex error that in the end tells me that

the kind of class I'm returning

```
function logger(target: any, ctx: ClassDecoratorContext) {
  return class extends target {
    age = 35;
  }
}
```

```
@logger
class Person {
  name = 'Max';
  greet() {
    console.log('Hi, I am ' + this.name);
  }
}
```

is not in line with the class I have here,

```
1 function logger(target: any, ctx: ClassDecoratorContext) {
2   console.log('logger decorator');
3   console.log(target);
4   console.log(ctx);
5
6   return class extends target {
7     age = 35;
8   }
9 }
10
11 @logger
12 class Person {
13   name = 'Max';
14
15   greet() {
16     console.log('Hi, I am ' + this.name);
```

What we should do instead is use a generic type,

```

1 function logger<T extends {}>(target: any, ctx: ClassDecoratorContext) {
2   console.log('logger decorator');
3   console.log(target);
4   console.log(ctx);
5
6   return class extends target {
7     age = 35;
8   }
9 }
10
11 @logger
12 class Person {
13   name = 'Max';
14
15   greet() {
16     console.log('it must extend some other type, a class type.

```

```

function logger<T extends >(target: any, ctx: ClassDecoratorContext) {
  console.log('logger decorator');
  console.log(target);
  console.log(ctx);

  return class extends target {
    age = 35;
  }
}

@logger
class Person {
  name = 'Max';

  greet() {

```

So the thing we're using logger on should be a valid class

and in TypeScript, you can express that by writing "new"

```

1 function logger<T extends new () => {}>(target: any, ctx: ClassDecorator
2   console.log('logger decorator');
3   console.log(target);
4   console.log(ctx);
5
6   return class extends target {
7     age = 35;
8   }
9 }
10
11 @logger
12 class Person {
13   name = 'Max';
14
15   greet() {

```

and then a function type thereafter.

but that's how you can tell TypeScript

that you want to interact with a class

```
1 function logger<T extends new () =>>(target: any, ctx: ClassDecoratorContext) {
2   console.log('logger decorator');
3   console.log(target);
4   console.log(ctx);
5
6   return class extends target {
7     age = 35;
8   }
9 }
10
11 @logger
12 class Person {
13   name = 'Max';
14
15   greet() {
16     console.log('hi, I am ' + this.name);
17   }
18 }
```

and then here, this function type in the end

represents the class constructor

and there, I want to be able to use my logger decorator

on any kind of class,

```
decorators.ts 3
1 function logger<T extends new (...args) =>>(target: any, ctx: ClassDecoratorContext) {
2   console.log('logger decorator');
3   console.log(target);
4   console.log(ctx);
5
6   return class extends target {
7     age = 35;
8   }
9 }
10
11 @logger
12 class Person {
13   name = 'Max';
14
15   greet() {
16     console.log('hi, I am ' + this.name);
17   }
18 }
```

so I'm fine with accepting any amount of arguments,

```

function logger<T extends new (...args: any[]) => any>(target: any, ctx: Context) {
  console.log('logger decorator');
  console.log(target);
  console.log(ctx);

  return class extends target {
    age = 35;
  }
}

@logger
class Person {
  name = 'Max';

  greet() {
    // ...
  }
}

```

which we can express like this,

That's what this syntax means.

```

TS decorators.ts 1
1 function logger<T extends new (...args: any[]) => any>(target: any, ctx: Context) {
2   console.log('logger decorator');
3   console.log(target);
4   console.log(ctx);
5
6   return class extends target {
7     age = 35;
8   }
9 }
10
11 @logger
12 class Person {
13   name = 'Max';
14
15   greet() {
16     console.log('Hello, my name is', this.name);
17   }
18 }

```

and it also will return any kind of value because again,

```

function logger<T extends new (...args: any[]) => any>(target: any, ctx: Context) {
  console.log('logger decorator');
  console.log(target);
  console.log(ctx);

  return class extends target {
    age = 35;
  }
}

@logger
class Person {
  name = 'Max';

  greet() {
    // ...
  }
}

```

I don't know in advance to which kind of class

```
x';
```

I want to attach this logger decorator.

```
1 function logger<T extends new (...args: any[]) => any>(target: any, ctx: any) {
2   console.log('logger decorator');
3   console.log(target);
4   console.log(ctx);
5
6   return class extends target {
7     age = 35;
8   }
9 }
10
11 @logger
12 class Person {
13   name = 'Max';
14
15   greet() {
16     // So that's how you can express
```

```
() {
  // that you wanna base your type "T" on some class
  // sole type that can be used as a base class
```

```
() that accepts any kind of arguments in its constructor
```

where you then return any kind of value

and now it's this "T" type, this generic type,

```
1 function logger<T extends new (...args: any[]) => any>(target: T, ctx: any) {
2   console.log('logger decorator');
3   console.log(target);
4   console.log(ctx);
5
6   return class extends target {
7     age = 35;
8   }
9 }
10
11 @logger
12 class Person {
13   name = 'Max';
14
15   greet() {
16     // which I use for target.
```

```
function logger<T extends new (...args: any[]) => any>(target: T, ctx:
  console.log('logger decorator');
  console.log(target);
  console.log(ctx);

  return class extends target {
    age = 35;
  }
}

@logger
class Person {
  name = 'Max';
  greet() {
    // this arrow down here goes away.
  }
}
```

```
function logger<T extends new (...args: any[]) => any>(target: T, ctx: ClassDecoratorContext)
  return class extends target{
    age = 30;
  }
}

@logger
class Person {
  name = 'Khalid';

  greet(){
    console.log('Hi, I am ' + this.name);
  }
}

+ constructor Person(): Person
const person = new Person();
console.log(person);

//in the terminal
// tsc
```

```
class Person {
  greet() {
    console.log('Hi, I am ' + this.name);
  }
}

const max = new Person();
```

PROBLEMS DEBUG CONSOLE PORTS COMMENTS

```
logger decorator
[class Person]
{
  kind: 'class',
  name: 'Person',
  metadata: undefined,
  addInitializer: [Function (anonymous)]
}
Person { name: 'Max', age: 35 }
```

```

10     age = 35;
11 };
12 }
13
14 @logger
15 class Person {
16     name = 'Max';
17
18     greet() {
19         console.log('Hi, I am ' + this.name);
20     }
21 }
22
23 // the original class does not get replaced by that class here,
24 const max = new Person();

```

{ but in the end, we merged them, you could say.

```

1 // Decorator is an object oriented related feature, I will start by adding a class
2
3 function logger<T extends new (...args: any[]) => any>(target: T, ctx: ClassDecoratorContext){
4     console.log('logger decorator');
5     console.log(target);
6     console.log(ctx);
7
8     return class extends target{
9         constructor(...args: any[]){
10             super(...args);
11             console.log('class constructor');
12             console.log(this);
13         }
14     }
15 }
16
17 @logger
18 class Person {
19     name = 'Khalid';
20
21     greet(){
22         console.log('Hi, I am ' + this.name);
23     }
24 }
25
26 const person = new Person();
27 const person2 = new Person();
28
29 //in the terminal
30 // tsc

```

138. Understanding Decorator Code Execution Order

```
19 class Person {
22   greet() {
24   }
25 }
26
27 const max = new Person();
28 const julia = new Person();
```

PROBLEMS DEBUG CONSOLE PORTS COMMENTS OUTPUT

```
kind: 'class',
name: 'Person',
metadata: undefined,
addInitializer: [Function (anonymous)]
}
```

```
class constructor
Person { name: 'Max' }
class constructor
Person { name: 'Max' }
```

And I'm getting them because I have my logs here

Creating a Method Decorator

```
decorator.ts X
decorator.ts > @logger
1 // Decorator is an object oriented related feature, I will start by adding a class
2
3 function logger<T extends new (...args: any[]) => any>(target: T) {
4   console.log('logger decorator');
5   console.log(target);
6
7   return class extends target {
8     constructor(...args: any[]) {
9       super(...args);
10      console.log('class constructor');
11      console.log(this);
12    }
13  }
14 }
15
16 @logger
17 class Person {
18   name = 'Khalid';
19
20   greet() {
21     console.log('Hi, I am ' + this.name);
22   }
23 }
24
25 const person = new Person();
26 const person2 = new Person();
27
28 //in the terminal
```

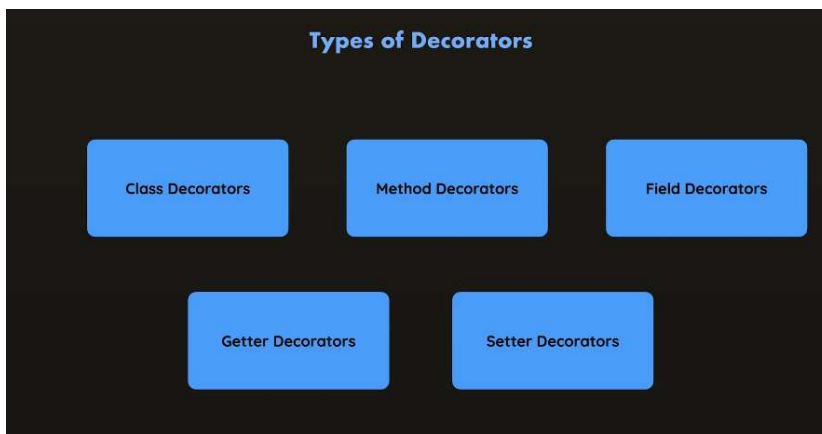
So that's how you can build
and use a decorator
and you can do anything you want with it in the end.

```
TS decorators.ts x
1 function logger<T extends new (...args: any[]) => any>({
2   target: T,
3   logger: Logger,
4   ctx: Context
5 }) {
6   console.log(target);
7   console.log(ctx);
8
9   return class extends target {
10     constructor(...args: any[]) {
11       super(...args);
12       console.log('class constructor');
13       console.log(this);
14     }
15   };
16 }
17
```

since you can return an updated version

since you can return an updated version of the thing you're attaching it to. You can use it to add more properties, methods, and that can make it a really powerful feature. You could, for example, create a decorator and share it as a library.

so that other developers can use it that enhances any kind of class it's added to by adding extra methods to it or anything like that. So that's really a powerful feature to consider and be aware of.



Here, however, I'll leave it at that and instead move on to a different kind of decorator you can build because as mentioned, you don't just have class decorators, instead you also have decorators that can be added to methods or fields.

So let's build a method decorator here,

```

decorators.ts M
1  function logger<T extends new (...args: any[]) => any>(
6  }
7
8  function autobind(
9      target: (...args: any[]) => any,
10     ctx: ClassMethodDecoratorContext
11 ) {}
12
13 @logger
14 class Person {
15     name = 'Max';
16
17     @autobind
18     greet() {
19         console.log('Hi, I am ' + this.name);

```

Creating a Method Decorator

```

File Edit Selection View Go Run Terminal Help
Section 11 - ECMAScript Decorators

decorators.ts M
1  // Decorator is an object oriented related feature, I will start by adding a class
2
3  function logger<T extends new (...args: any[]) => any>(target: T) {
4      console.log('logger decorator');
5      console.log(target);
6
7      return class extends target {
8          constructor(...args: any[]) {
9              super(...args);
10             console.log('class constructor');
11             console.log(this);
12         }
13     }
14 }
15
16 function autobind(target: (...args: any[]) => any, ctx: ClassMethodDecoratorContext) {
17     console.log(target);
18     console.log(ctx);
19 }
20
21 @logger
22 class Person {
23     name = 'Khalid';
24
25     @autobind
26     greet() {
27         console.log('Hi, I am ' + this.name);
28     }
29 }
30
31 const person = new Person();
32 const person2 = new Person();
33
34 //in the terminal
35
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Practice\TypeScript\Section 11 - ECMAScript Decorators> node decorator.js
{
  kind: 'method',
  name: 'greet',
  static: false,
  private: false,
  access: { has: [Function: has], get: [Function: get] },
  metadata: undefined,
  addInitializer: [Function (anonymous)]
}
logger decorator
[class Person]
class constructor
Person { name: 'Khalid' }
class constructor
Person { name: 'Khalid' }
PS D:\Practice\TypeScript\Section 11 - ECMAScript Decorators>

```

but it's the very first log that's my method decorator here because the method gets initialized before the class initialization is done.

That's why the decorator added to that method executes before the class decorator executes because all these decorators only execute once the thing they have been attached to is done initializing. And you see here, it's the greet function,

the greet method that's being logged.

Using Decorators to Solve A Common Problem


```

27  class Person {
28
29
30      @autobind
31      greet() {
32          console.log('Hi, I am ' + this.name);
33      }
34  }
35
36  const max = new Person();
37  const greet = max.greet;
38  greet();

```

If I run and compile this code

```

$ tsc
$ node decorators.js

```

I am getting an error here

PROBLEMS DEBUG CONSOLE PORTS COMMENTS OUTPUT TERMINAL

```

console.log('Hi, I am ' + this.name);
                        ^

```

```

TypeError: Cannot read properties of undefined (reading 'name')
    at greet (/Users/max/development/teaching/understanding-typescript/decorators/decorators.js:72:44)
    at Object.<anonymous> (/Users/max/development/teaching/understanding-typescript/decorators/decorators.js:79:1)

```

What's the problem here? The problem is related to 'this' keyword in JavaScript. It's not typescript specific at all.

```

27  class Person {
28
29
30      @autobind
31      greet() {
32          console.log('Hi, I am ' + this.name);
33      }
34  }
35
36  const max = new Person();
37  const greet = max.greet;
38  max.greet();
39  greet();

```

This is ok because greet is being executed on max.

then greet is executed on max.

```

5
6  const max = new Person();
7  const greet = max.greet;
8  greet();

```

when executed like this, then
greet is not directly executed on something

You can solve this problem by binding this class.

```

24  }
25
26  @logger
27  class Person {
28      name = 'Max';
29
30      constructor() {
31          this.greet = this.greet.bind(this);
32      }
33
34      @autobind
35      greet() {
36          console.log('Hi, I am ' + this.name);
37      }
38  }
39

```

but with that you're saying,

JS has a solution for that but it is kinda annoying.

Because in that case, you have to do it for every function in the constructor.

That's when decorator can solve this problem

Implementing A Decorator-based Solution: autobind

let's solve it with this decorator.
Because my idea is to put that code
which I had in the constructor here in this person class
in the last lecture into this decorator.

So that this decorator will automatically bind the method
it's attached to, to the class the method belongs to.

```

17
18 function autobind(
19   target: (...args: any[]) => any,
20   ctx: ClassMethodDecoratorContext
21 ) {
22   ctx.addInitializer();
23 }
24
25 @logger
26 class Person {
27   name = 'Max';
28
29   @autobind
30   greet() {
31     console.log('Hi, I am ' + this.name);
32   }

```



`.log('Hi, I am ' + this.name);`
to allow you to run code related to this thing

you are attaching the decorator to

after the surrounding thing.

so the surrounding class is done initializing.

So in the end you can think

of this addInitializer method here as a way

of giving you access to the constructor of the class

```

function autobind(
  target: (...args: any[]) => any,
  ctx: ClassMethodDecoratorContext
) {
  ctx.addInitializer();
}

@logger
class Person {
  name = 'Max';

  @autobind
  greet() {
    console.log('Hi, I am ' + this.name);
  }

```

the method this decorator is added to belongs to.

```

8 function autobind(
9   target: (...args: any[]) => any,
10  ctx: ClassMethodDecoratorContext
11 ) {
12   ctx.addInitializer(function() {});
13 }
14
15 @logger
16 class Person {
17   name = 'Max';
18
19   @autobind
20   greet() {
21     console.log('For that you must pass a function to addInitializer');
22   }
23 }

```

Introducing Field Decorator

```

TS decorators.ts
18 function autobind(
27   console.log('Executing original function');
28   target.apply(this);
29 }
30
31
32 function fieldLogger(target: undefined, ctx: ClassFieldDecoratorContext) {
33   // ...
34 }
35
36 @logger
37 class Person {
38   @fieldLogger
39   name = 'Max';
40
41   @autobind

```

So this was a method decorator.
The last kind of decorator I wanna explore together with you is the field decorator,

so a decorator that can be added to fields like name here.
And for that, I'll create another new function because a decorator is still a function and I'll simply name it fieldLogger because I wanna keep this decorator simple and just log something again.
Now, like before, we add it with the @ symbol, and like before,

we must accept the appropriate amount of arguments.
So for that here, we need a target and a context.
And the type of target here for fields will actually always be undefined because the decorator code will be executed before the field is done initializing.
That's important.

That's why the target, the value of the field, will be undefined.
The value of the greet method was the function, the actual method function.
The value of the class for the class decorator was the finished class.
But the value, the target of the field decorator, fieldLogger here will be before it's been initialized and it will therefore always be undefined.

That's just something to keep in mind.
When creating an ECMAScript field decorator, target will be undefined.
The context, however, will not be undefined.
Instead that will be a ClassFieldDecoratorContext object.

```
//Field Decorator
function fieldLogger(target: undefined, ctx: ClassFieldDecoratorContext){
  console.log(target);
  console.log(ctx);
}

@logger
class Person{
  @fieldLogger

  name = 'Khalid';

  @autobind
  greet(){
    console.log('Hi, I am ' + this.name);
  }
}
```

PROBLEMS DEBUG CONSOLE PORTS COMMENTS OUTPUT

- \$ tsc
- \$ node decorators.js

- \$ node decorators.js

```
undefined
```

```
{
  kind: 'field',
  name: 'name',
  static: false,
  private: false,
  access: { has: [Function: has], get: [Function: get], set: [Function: set]
}
```

undefined is logged, as I explained.

```
31
32 function fieldLogger(target: undefined, ctx: ClassFieldDecoratorContext)
33   console.log(target);
34   console.log(ctx);
35 }
36
37 @logger
38 class Person {
39   @fieldLogger
40   name = 'Max';
41
42   @autobind
43   greet() {
```

Now, just as with class and method decorators,

```
function fieldLogger(target: undefined, ctx: ClassFieldDecoratorContext)
  console.log(target);
  console.log(ctx);

  return
}
```

```
@logger
class Person {
  @fieldLogger
  name = 'Max';
```

you can also return something here

to change the thing you're attaching the decorator to.

So to change the field or to be precise, it's value.

143. Introducing the Field Decorator

```
31
32 function fieldLogger(target: undefined, ctx: ClassFieldDecoratorContext)
33   console.log(target);
34   console.log(ctx);
35
36   return (initialValue: any) => {
37     console.log(initialValue);
38     return ' ';
39   }
40 }
41
42 @logger
43 class Person {
```

143. Introducing the Field Decorator

```
32 function fieldLogger(target: undefined, ctx: ClassFieldDecoratorContext)
34   console.log(ctx);
35
36   return (initialValue: any) => {
37     console.log(initialValue);
```

143. Introducing the Field Decorator

```
32 function fieldLogger(target: undefined, ctx: ClassFieldDecorator  
34     console.log(ctx);  
35  
36     return (initialValue: any) => {  
37         console.log(initialValue);  
38         return '';  
39     }  
40 }
```

PROBLEMS DEBUG CONSOLE PORTS COMMENTS OUTPUT TERMINAL

```
name: 'Person',  
metadata: undefined,  
addInitializer: [Function (anonymous)]  
}  
Max  
class constructor  
Person { greet: [Function: bound ], name: '' }  
Executing original function because I changed the field value to an empty string.  
Hi, I am
```

Building Configurable Decorators with Factories

And a decorator factory in the end is just a function
that produces a decorator.
And why would you do that?
Well, let's say here for my field logger decorator,
where I'm changing the default value,
I actually want to be able to set the default value.